

Assignment: Inference Engineering

Week Task

Part A – Theory

1. Difference between latency and throughput in model inference; which factor influences performance more, and how?

Latency: The time a model takes to give a response to a single request. It is based on a request–response cycle – the time it takes for data to travel from one node to another.

Example: For a single request, a model takes 500 ms to give a response.

$$\text{Latency} = 500 \text{ ms}$$

Throughput: The number of requests (or tokens) a model can process in a given amount of time. It is based on how much data can be processed within a fixed time window.

Example: A model named “Ravan” processes 1000 requests per second.

$$\text{Throughput} = 1000 \text{ req/sec}$$

Which factor influences performance more? This depends on the application being used:

- **Chatbot** → Latency matters more → users expect an immediate response.
- **Voice assistant** → Latency matters more → any delay feels annoying.

- **High-traffic serving systems** → Throughput matters more → need to handle many requests at once.
- **Interactive LLM applications** → Both matter → need fast responses *and* support for many concurrent users.

2. What is KV cache and how does it help with model generation?

The **KV Cache (Key–Value Cache)** is a temporary storage mechanism used in inference engines to avoid repetitive computation. It stores the Key and Value vectors for tokens that have already been processed, so they do not need to be recalculated.

Example:

1. The model processes “What” → stores its K and V.
2. Processes “is” → stores its K and V.
3. Processes “AI” → stores its K and V.
4. When generating the next token, it reuses the stored K and V instead of recalculating them.

Previous tokens → K/V stored in cache → Reused during next-token generation

In summary, the KV Cache stores the Key and Value representations of previously processed tokens during autoregressive generation, allowing the model to reuse them instead of recomputing them. This significantly improves inference speed at the cost of additional memory usage.

3. What is quantization, and what is the trade-off between INT8/INT4 quantization and model accuracy? Give an example with calculations.

Quantization is the process of converting high-precision model weights into a lower-precision format – for example, converting FP32 weights into a lower-precision floating point or integer format. It reduces model size and can improve inference speed, making it easier to run large models on smaller, memory-constrained hardware.

Trade-off:

- **INT8** uses 8 bits (1 byte) per weight. It is more accurate than INT4 because it retains more precision.
- **INT4** uses 4 bits per weight. It gives much greater memory savings but can cause more accuracy loss.

Example – memory savings for a 7-billion parameter model:

FP16: $7\text{B} \times 16 \text{ bits} = 14 \text{ GB}$

INT8: $7\text{B} \times 8 \text{ bits} = 7 \text{ GB}$

INT4: $7\text{B} \times 4 \text{ bits} = 3.5 \text{ GB}$

Quantization – Scale and Zero Point

Quantization converts a floating-point value into a smaller integer value, such as INT8, using the formula:

$$X_q = \text{round} \left(\frac{X}{S} \right)$$

where:

- X_q = Quantized value
- X = Actual / original value
- S = Scale

Example:

Given: $X = 0.72$, $S = 0.0078$, Zero Point (Z) = 0

$$X_q = \text{round} \left(\frac{0.72}{0.0078} \right) = \text{round}(92.30) = 92$$

Therefore, the original value 0.72 (FP32) is stored as the quantized INT8 value 92.

4. Name one real inference serving framework and briefly describe one optimization it does.

vLLM is an open-source inference serving framework designed for Large Language Models (LLMs). It helps deploy models and efficiently handle multiple concurrent user requests.

Optimization: PagedAttention

vLLM uses **PagedAttention** to efficiently manage the KV Cache during inference. Instead of storing the KV Cache in one large contiguous block of memory, it divides the cache into smaller blocks (“pages”). These blocks can be allocated and reused efficiently across different requests, which reduces GPU memory wastage and allows vLLM to serve more requests at the same time.

Benefits:

- Better GPU memory utilization
- Supports more concurrent requests
- Reduces KV Cache memory wastage
- Improves inference throughput
- Helps serve multiple users efficiently

Example:

- **Without PagedAttention:** Multiple requests → KV Cache stored contiguously → memory wastage.
- **With PagedAttention:** Multiple requests → KV Cache divided into blocks → efficient memory usage → more requests served.